

VaultScan

How Our Product Detects Cloud Misconfigurations — and Why You Can Trust the Results

1. What VaultScan is

VaultScan is a **Cloud Security Posture Management (CSPM)** product. It does not attack your cloud and it does not replace firewalls or antivirus. Instead, it acts like a continuous **security building inspector**: it reads the configuration of your AWS account and flags settings that are known to cause breaches — public storage, open admin ports, missing encryption, over-privileged identities, and related gaps.

Misconfigurations are not theoretical. They are one of the most common root causes of cloud data exposure: attackers do not always need a zero-day exploit when a bucket or security group is left open to the internet.

2. How detection works (end to end)

Step A — Secure connection (industry standard)

VaultScan connects using the same model used by enterprise CSPM platforms (Wiz, Prisma Cloud, Orca style):

1. You create (or select) an IAM user or identity that can call **sts:AssumeRole**.
2. You create a **read-only IAM Role** in the target account with a trust policy that allows that identity.
3. VaultScan stores connection settings server-side (Settings UI) and assumes the role for a short-lived session.
4. All discovery APIs run under the **role credentials**, not permanent admin keys.

That means: least privilege, easy revocation (delete the role or trust), and no need to store target-account admin secrets for every scan.

Step B — Live API inventory (not guesswork)

When you click **Launch active scan**, the backend opens a boto3 session (after AssumeRole or direct credentials / demo mode) and calls real AWS APIs. It does **not** invent resources. If AWS returns a bucket list, IAM users, security groups, or databases, those are the objects evaluated.

Supported services in the current engine include:

- **S3** — ACLs, public access block, versioning, default encryption, public bucket policies
- **IAM** — admin policies, wildcard inline policies, MFA, old access keys, roles with AdministratorAccess
- **EC2** — security groups open to 0.0.0.0/0 (SSH/RDP/DB ports/all traffic), unencrypted EBS, IMDSv1
- **RDS** — publicly accessible instances, storage encryption, backup retention

Step C — Deterministic rules (same input → same finding)

Each check is a fixed rule with a clear condition, severity, compliance mapping, and remediation. Example logic:

- If a security group allows TCP port **22 from 0.0.0.0/0** → CRITICAL finding (internet-wide SSH).
- If a bucket grants ACL to AllUsers → CRITICAL public exposure.
- If an IAM user has no MFA devices → HIGH identity risk.
- If default encryption is missing on S3 → HIGH data-at-rest risk.

Rules are implemented in code (**scanner_engine.py**), not as opaque ML guesses. That is important for trust: auditors and engineers can reason about *why* a finding appeared.

Step D — Severity scoring & posture score

Findings are labeled CRITICAL / HIGH / MEDIUM / LOW. A posture score starts at 100 and subtracts weighted penalties (critical issues cost more than low ones). The score is a prioritization aid, not a marketing badge — it moves when you fix real configurations and re-scan.

Step E — Compliance mapping

Each finding carries framework tags (e.g. CIS AWS, NIST 800-53, GDPR Art. 32, HIPAA). The dashboard aggregates these into a compliance snapshot so security and leadership share one language.

Step F — Explainability (Cloud Assistant & reports)

Cloud Assistant and PDF/Word reports explain findings in plain language and propose remediations (often exact AWS CLI). AI is used for **explanation and prioritization narrative**. It does not invent AWS resources: it is grounded in the scan JSON produced by the rule engine.

3. Why you can trust the results

Trust factor	What VaultScan does
Read-only by design	Uses AssumeRole + audit-style APIs. Scanning does not mutate your cloud unless you explicitly run remediation outside the product.
Evidence-based	Findings come from AWS API responses (ACLs, policies, SG rules, encryption settings) — not synthetic random alerts.
Deterministic rules	Same configuration yields the same findings. Transparent severity and rule IDs.
Industry benchmarks	Checks align with CIS AWS Foundations and common NIST / privacy expectations.
Least-privilege auth	Connector keys only need sts:AssumeRole; the role can be scoped and revoked anytime.
Graceful access control	If a service is denied (AccessDenied), VaultScan reports that honestly instead of pretending the service is clean.
Reproducible history	Scans and reports persist so you can re-open and export without inventing a new scan.
Separation of concerns	Rule engine detects; Cloud Assistant explains; humans approve fixes.

4. What VaultScan is not (honest limits)

- **Not a vulnerability scanner for OS packages** (no CVE deep dive into EC2 guest OS by default).
- **Not runtime threat detection** (no continuous packet inspection / EDR).
- **Not a guarantee of zero risk** — unknown zero-days and social engineering are out of scope.
- **Depends on permissions** — if the role cannot describe a service, that service cannot be fully assessed.
- **Demo mode** uses simulated resources for training; real trust comes from AssumeRole against your account.

Trust improves when you attach the recommended read-only policies, scan production-like accounts, and re-scan after remediation to prove posture score and findings improve.

5. Example: from API to finding

Input (AWS): `ec2.describe_security_groups()` returns a rule `IpProtocol=tcp, FromPort=22, CidrIp=0.0.0.0/0`.

Rule: Dangerous admin port open to the world → severity CRITICAL, rule id like SG-OPEN-22.

Output: Finding with resource `sg-...`, human description, CIS mapping, and remediation `aws ec2 revoke-security-group-ingress ...`

Anyone on your team can reproduce this with the AWS CLI/console. That reproducibility is the core of technical trust.

6. Product architecture (simple)

Layer	Component	Responsibility
UI	Next.js dashboard	Settings, scan launch, findings, reports, Cloud Assistant
API	FastAPI backend	Auth to AWS, run engine, store scans, export PDF/DOCX
Engine	<code>scanner_engine.py</code>	boto3 checks + scoring + compliance tags
AWS	STS + service APIs	Source of truth for configuration state

Assist	Cloud Assistant	Explain findings; grounded on scan data
--------	-----------------	---

7. How to verify yourself (recommended)

1. Connect a non-production account via Settings (AssumeRole).
2. Intentionally create a known-bad control (e.g. public S3 ACL or open SSH SG) in a sandbox.
3. Launch scan — VaultScan should surface that exact resource.
4. Fix the setting in AWS console/CLI.
5. Re-scan — finding should disappear or severity reduce; posture score should improve.
6. Export PDF report and confirm the same findings appear with remediations.

If a finding cannot be reproduced in AWS, treat it as a product bug and investigate. That closed loop is how teams build operational trust.